IUT Saint-Dié-des-Vosges

UNIVERSITÉ DE LORRAINE

Bases de traitements d'images avec OpenCV
TP2

Hugo Rositi*

15 octobre 2026

---

1 OpenCV JS

Réaliser des opérations de base de traitements d'images :

1. Chargement d'images
2. Changement d'espaces couleurs
3. Seuillage interactif avec slider

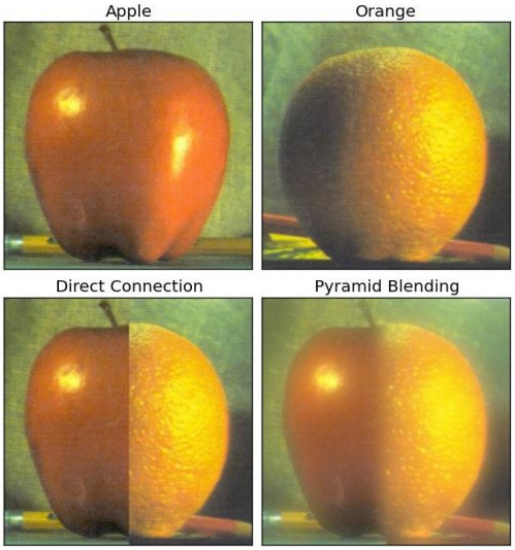
En ayant recours à la bibliothèque OpenCV dans son implémentation en Javascript.
Vous récupérerez la bibliothèque à travers la documentation.

2 Détection de contours

1. Expérimenter les différents filtres pour détecter les contours d'une image.
   - Prewitt
   - Sobel
   - Canny
2. Trouver ensuite le moyen de d'afficher le contour connexe le plus long (`cv.findContours()`).

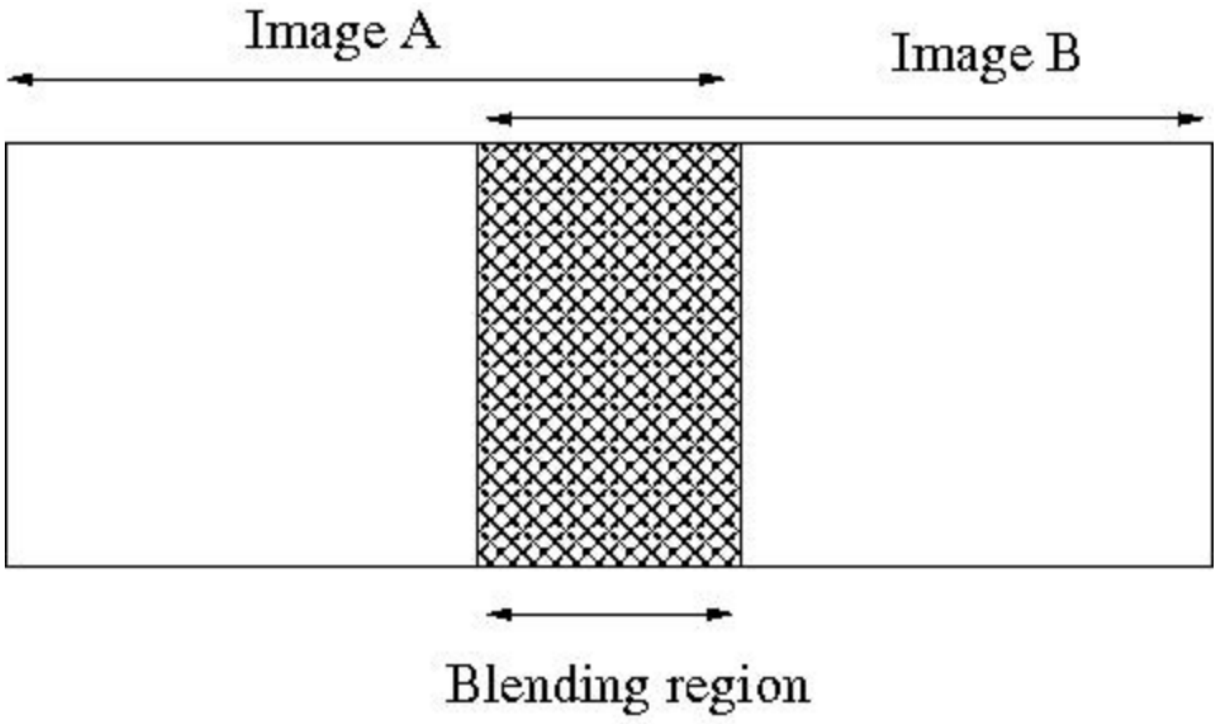
*hugo.rositi@univ-lorraine.fr

3 Fusion d'images



3.1 Fusion d'images par pondération directe

$$PB(i,j) = (1-w)*PA(i,j) + w*PB(i,j)$$



1. En appliquant la formule de pondération directe, réaliser une fusion de deux images.

3.2 Fusion d'images par l'usage de pyramide multi-résolution

1. Charger les deux images de pomme et d'orange.
2. Trouver les pyramides Gaussiennes de ces deux images (le nombre de niveau est ici de 6)
3. À partir des pyramides Gaussiennes, trouver les pyramides laplaciennes.
4. Joindre les moitiés gauche de la pomme et droite de l'orange à chaque niveau de la pyramide laplacienne.
5. Enfin, à partir de cette pyramide jointe, reconstruire l'image originale

4 Détection par la couleur

Créer une application simple qui permet de segmenter un objet dans une image par sa couleur. La figure 1 montre un exemple où une voiture jaune a été sélectionnée. Vous devez utiliser des trackbars pour permettre à l'utilisateur de choisir la couleur de l'objet, dans l'espace de couleur HSV. Voici un algorithme possible :

1. Convertir l'image d'entrée dans l'espace couleur HSV
2. A l'aide de 6 trackbars obtenir la couleur min et max.
3. Segmenter l'image d'entrée dans l'intervalle $[MIN, MAX]$
4. Utiliser l'opérateur AND_Logique entre l'image d'entrée et le mask

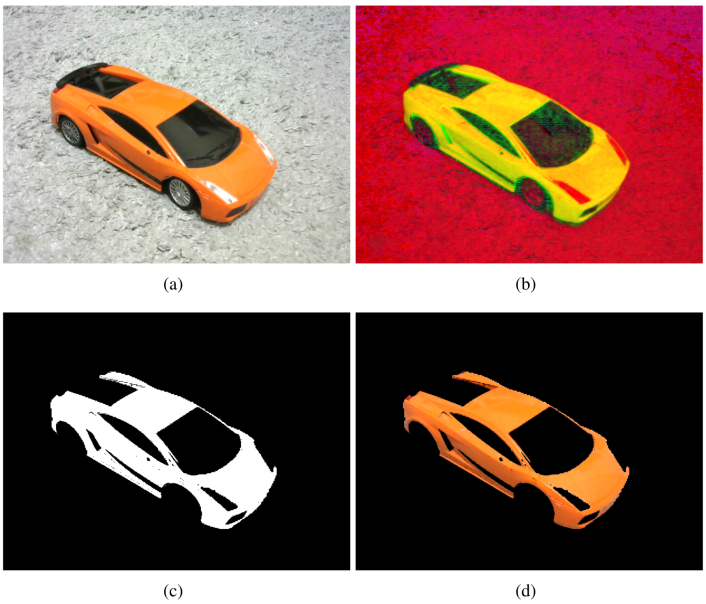


FIGURE 1 – (a) image input en RGB, (b) image input en HSV, (c) mask binaire, (d) résultat de la segmentation

5 Détection d'objets géométriques

Créer une application simple dont le pipeline est illustré sur la figure 2. Voici un algorithme possible :

1. Transformer l'image d'entrée en niveaux de gris
2. Réduire le bruit à l'aide d'un filtre gaussien
3. Extraire les pixels des contours avec un filtre de Canny
4. Trouver les contours Rq : contours est une liste Python de tous les contours de l'image. Chaque contour individuel est un tableau Numpy de coordonnées (x, y) des points limites (sur le contour) de l'objet.
5. Afficher les contours trouvés
6. Calculer la boîte englobante et le cercle circonscrit de chaque contour trouvé
7. Calculer une approximation polygonale avec le minimum de sommets
8. Utiliser le nombre de coins pour étiqueter les formes

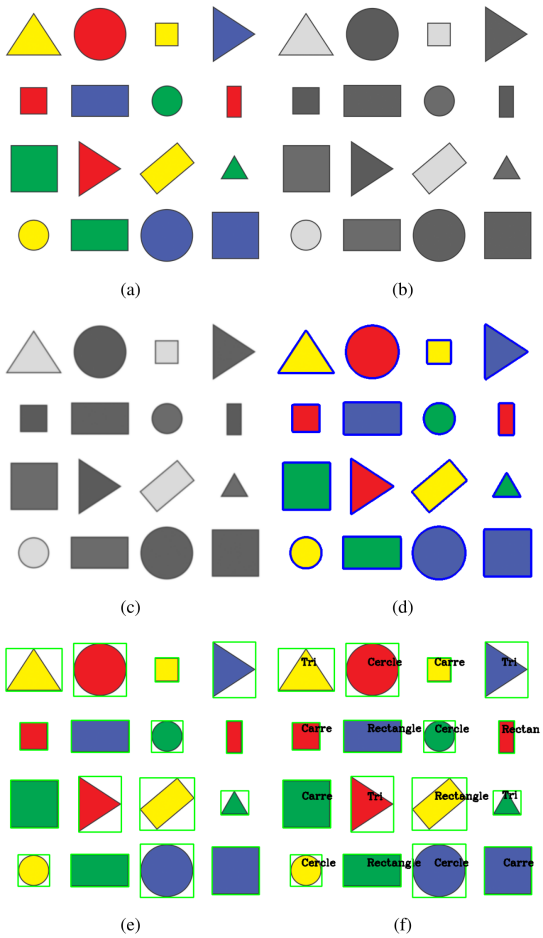


FIGURE 2 – (a) image input, (b) image input en niveaux de gris, (c) image lissée, (d) contours, (e) boîtes englobantes, (f) objets reconnus avec labels